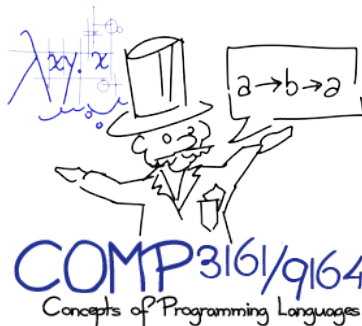




Functional Programming Languages

Johannes Åman Pohjola
UNSW
Term 3 2023

Functional Programming

Many languages have been called **functional** over the years:

Lisp

```
(define (max-of lst)  
  (cond  
    [(= (length lst) 1) (first lst)]  
    [else (max (first lst) (max-of (rest lst)))]))
```

Functional Programming

Many languages have been called **functional** over the years:

Haskell

```
maxOf :: [Int] → Int  
maxOf = foldr1 max
```

Lisp

```
(define (max-of lst)  
  (cond  
    [(= (length lst) 1) (first lst)]  
    [else (max (first lst) (max-of (rest lst)))]))
```

Functional Programming

Many languages have been called **functional** over the years:

Haskell

```
maxOf :: [Int] → Int  
maxOf = foldr1 max
```

Lisp

```
(define (max-of lst)  
  (cond  
    [(= (length lst) 1) (first lst)]  
    [else (max (first lst) (max-of (rest lst)))]))
```

JavaScript?

```
function maxOf(arr) {  
  var max = arr.reduce(function(a, b) {  
    return Math.max(a, b);  
  });  
}
```

Functional Programming

Many languages have been called **functional** over the years:

Haskell

```
maxOf :: [Int] → Int  
maxOf = foldr1 max
```

Lisp

```
(define (max-of lst)  
  (cond  
    [(= (length lst) 1) (first lst)]  
    [else (max (first lst) (max-of (rest lst)))]))
```

JavaScript?

```
function maxOf(arr) {  
  var max = arr.reduce(function(a, b) {  
    return Math.max(a, b);  
  });  
}
```

What do they
have in **common**?

Definitions

Unlike imperative languages, **functional** programming languages are not very crisply defined.

Attempt at a Definition

A *functional programming language* is a programming language derived from or inspired by the λ -calculus, or derived from or inspired by another functional programming language.

The result? If it has λ in it, you can call it functional.

Definitions

Unlike imperative languages, **functional** programming languages are not very crisply defined.

Attempt at a Definition

A *functional programming language* is a programming language derived from or inspired by the λ -calculus, or derived from or inspired by another functional programming language.

The result? If it has λ in it, you can call it functional.

In this course, we'll consider *purely functional* languages, which have a much better definition.

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Garbage Collection?

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Garbage Collection?

Lisp, 1958

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Garbage Collection?

Lisp, 1958

Functions as Values?

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Garbage Collection?

Lisp, 1958

Functions as Values?

Lisp, 1958

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Garbage Collection?

Lisp, 1958

Polymorphism?

Functions as Values?

Lisp, 1958

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Garbage Collection?

Lisp, 1958

Polymorphism?

ML, 1973

Functions as Values?

Lisp, 1958

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Type Inference?

Garbage Collection?

Lisp, 1958

Polymorphism?

ML, 1973

Functions as Values?

Lisp, 1958

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Type Inference?

ML, 1973

Garbage Collection?

Lisp, 1958

Polymorphism?

ML, 1973

Functions as Values?

Lisp, 1958

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Type Inference?

ML, 1973

Garbage Collection?

Lisp, 1958

Metaprogramming?

Polymorphism?

ML, 1973

Functions as Values?

Lisp, 1958

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Type Inference?

ML, 1973

Garbage Collection?

Lisp, 1958

Metaprogramming?

Lisp, 1958

Polymorphism?

ML, 1973

Functions as Values?

Lisp, 1958

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Type Inference?

ML, 1973

Garbage Collection?

Lisp, 1958

Metaprogramming?

Lisp, 1958

Polymorphism?

ML, 1973

Functions as Values?

Lisp, 1958

Lazy Evaluation?

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Type Inference?

ML, 1973

Garbage Collection?

Lisp, 1958

Metaprogramming?

Lisp, 1958

Polymorphism?

ML, 1973

Functions as Values?

Lisp, 1958

Lazy Evaluation?

Miranda, 1985

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Monads?

Type Inference?

ML, 1973

Garbage Collection?

Lisp, 1958

Metaprogramming?

Lisp, 1958

Polymorphism?

ML, 1973

Functions as Values?

Lisp, 1958

Lazy Evaluation?

Miranda, 1985

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Monads?

Haskell, 1991

Type Inference?

ML, 1973

Garbage Collection?

Lisp, 1958

Metaprogramming?

Lisp, 1958

Polymorphism?

ML, 1973

Functions as Values?

Lisp, 1958

Lazy Evaluation?

Miranda, 1985

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Monads?

Haskell, 1991

Type Inference?

ML, 1973

Garbage Collection?

Lisp, 1958

Software Transactional Memory?

Metaprogramming?

Lisp, 1958

Polymorphism?

ML, 1973

Functions as Values?

Lisp, 1958

Lazy Evaluation?

Miranda, 1985

Why Study FP Languages?

Think of a major innovation in the area of programming languages.

Monads?

Haskell, 1991

Type Inference?

ML, 1973

Garbage Collection?

Lisp, 1958

Software Transactional Memory?

GHC Haskell, 2005

Metaprogramming?

Lisp, 1958

Polymorphism?

ML, 1973

Functions as Values?

Lisp, 1958

Lazy Evaluation?

Miranda, 1985

Purely Functional Programming Languages

The term *purely functional* has a very crisp definition.

Definition

A programming language is *purely functional* if β -reduction (or evaluation in general) is actually a **confluence**.

In other words, functions have to be mathematical functions, and free of *side effects*.

Purely Functional Programming Languages

The term *purely functional* has a very crisp definition.

Definition

A programming language is *purely functional* if β -reduction (or evaluation in general) is actually a **confluence**.

In other words, functions have to be mathematical functions, and free of *side effects*.

Consider what would happen if we allowed effects in a functional language:

```
count = 0;  
f x = {count := count + x; return count};  
m = ( $\lambda y. y + y$ ) (f 3)
```

If we evaluate $f\ 3$ first, we will get $m = 6$, but if we β -reduce m first, we will get $m = 9$. $\Rightarrow$ **not confluent**.

Making a Functional Language

We're going to make a language called **MinHS**.

- 1 Three types of values: integers, booleans, and **functions**.

Making a Functional Language

We're going to make a language called **MinHS**.

- ① Three types of values: integers, booleans, and **functions**.
- ② Static type checking (not inference)

Making a Functional Language

We're going to make a language called **MinHS**.

- ① Three types of values: integers, booleans, and **functions**.
- ② Static type checking (not inference)
- ③ Purely functional (no effects)

Making a Functional Language

We're going to make a language called **MinHS**.

- 1 Three types of values: integers, booleans, and **functions**.
- 2 Static type checking (not inference)
- 3 Purely functional (no effects)
- 4 Call-by-value (strict evaluation)

Making a Functional Language

We're going to make a language called **MinHS**.

- ① Three types of values: integers, booleans, and **functions**.
- ② Static type checking (not inference)
- ③ Purely functional (no effects)
- ④ Call-by-value (strict evaluation)

Something not unlike this will appear in your **Assignment 1**.

Syntax

<i>Integers</i>	n	$::=$	$\dots$
<i>Identifiers</i>	f, x	$::=$	$\dots$
<i>Literals</i>	b	$::=$	$\text{True} \mid \text{False}$
<i>Types</i>	τ	$::=$	$\text{Bool} \mid \text{Int} \mid \tau_1 \rightarrow \tau_2$
<i>Infix Operators</i>	$\otimes$	$::=$	$* \mid + \mid == \mid \dots$
<i>Expressions</i>	e	$::=$	$x \mid n \mid b \mid (e) \mid e_1 \otimes e_2$ $\mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3$

Syntax

<i>Integers</i>	n	$::=$	$\dots$
<i>Identifiers</i>	f, x	$::=$	$\dots$
<i>Literals</i>	b	$::=$	$\text{True} \mid \text{False}$
<i>Types</i>	τ	$::=$	$\text{Bool} \mid \text{Int} \mid \tau_1 \rightarrow \tau_2$
<i>Infix Operators</i>	$\otimes$	$::=$	$* \mid + \mid == \mid \dots$
<i>Expressions</i>	e	$::=$	$x \mid n \mid b \mid (e) \mid e_1 \otimes e_2$ $\mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3$ $\mid e_1 \ e_2$

Syntax

<i>Integers</i>	n	$::=$	$\dots$
<i>Identifiers</i>	f, x	$::=$	$\dots$
<i>Literals</i>	b	$::=$	$\text{True} \mid \text{False}$
<i>Types</i>	τ	$::=$	$\text{Bool} \mid \text{Int} \mid \tau_1 \rightarrow \tau_2$
<i>Infix Operators</i>	$\otimes$	$::=$	$* \mid + \mid == \mid \dots$
<i>Expressions</i>	e	$::=$	$x \mid n \mid b \mid (e) \mid e_1 \otimes e_2$ $\mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3$ $\mid e_1 \ e_2$ $\mid \text{recfun } f :: (\tau_1 \rightarrow \tau_2) \ x = e$ $\uparrow \text{ Like } \lambda, \text{ but with recursion.}$

As usual, this is **ambiguous** concrete syntax. But all the precedence and associativity rules apply as in Haskell. We assume a suitable parser.

Examples

Example (Stupid division by 5)

```
recfun divBy5 :: (Int → Int) x =  
  if x < 5  
    then 0  
    else 1 + divBy5 (x - 5)
```

Example (Average Function)

```
recfun average :: (Int → (Int → Int)) x =  
  recfun avX :: (Int → Int) y =  
    (x + y) / 2
```

As in Haskell, $(\text{average } 15 \ 5) = ((\text{average } 15) \ 5)$.

We don't need no **let**

This language is so minimal, it doesn't even need **let** expressions.
How can we do without them?

We don't need no let

This language is so minimal, it doesn't even need **let** expressions.
How can we do without them?

$$\mathbf{let} \ x :: \tau_1 = e_1 \ \mathbf{in} \ e_2 :: \tau_2 \quad \equiv \quad (\mathbf{recfun} \ f :: (\tau_1 \rightarrow \tau_2) \ x = e_2) \ e_1$$

Abstract Syntax

Moving to **first order** abstract syntax, we get:

Concrete Syntax	Abstract Syntax
n	<code>(Num n)</code>
b	<code>(Lit n)</code>

Abstract Syntax

Moving to **first order** abstract syntax, we get:

Concrete Syntax	Abstract Syntax
n	$(\text{Num } n)$
b	$(\text{Lit } n)$
if c then t else e	$(\text{If } c \ t \ e)$

Abstract Syntax

Moving to **first order** abstract syntax, we get:

Concrete Syntax	Abstract Syntax
n	$(\text{Num } n)$
b	$(\text{Lit } n)$
if c then t else e	$(\text{If } c \ t \ e)$
$e_1 \ e_2$	$(\text{Apply } e_1 \ e_2)$

Abstract Syntax

Moving to **first order** abstract syntax, we get:

Concrete Syntax	Abstract Syntax
n	(Num n)
b	(Lit n)
if c then t else e	(If c t e)
e_1 e_2	(Apply e_1 e_2)
recfun $f :: (\tau_1 \rightarrow \tau_2) x = e$	(Recfun τ_1 τ_2 f x e)

Abstract Syntax

Moving to **first order** abstract syntax, we get:

Concrete Syntax	Abstract Syntax
n	(Num n)
b	(Lit n)
if c then t else e	(If c t e)
e_1 e_2	(Apply e_1 e_2)
recfun $f :: (\tau_1 \rightarrow \tau_2)$ $x = e$	(Recfun τ_1 τ_2 f x e)
x	(Var x)

Abstract Syntax

Moving to **first order** abstract syntax, we get:

Concrete Syntax	Abstract Syntax
n	(Num n)
b	(Lit n)
if c then t else e	(If c t e)
e_1 e_2	(Apply e_1 e_2)
recfun $f :: (\tau_1 \rightarrow \tau_2)$ $x = e$	(Recfun τ_1 τ_2 f x e)
x	(Var x)

What changes when we move to **higher order** abstract syntax?

Abstract Syntax

Moving to **first order** abstract syntax, we get:

Concrete Syntax	Abstract Syntax
n	$(\text{Num } n)$
b	$(\text{Lit } n)$
if c then t else e	$(\text{If } c \ t \ e)$
$e_1 \ e_2$	$(\text{Apply } e_1 \ e_2)$
recfun $f :: (\tau_1 \rightarrow \tau_2) \ x = e$	$(\text{Recfun } \tau_1 \ \tau_2 \ f \ x \ e)$
x	$(\text{Var } x)$

What changes when we move to **higher order** abstract syntax?

- 1 **Var** terms go away – we use the meta-language's variables.
- 2 $(\text{Recfun } \tau_1 \ \tau_2 \ f \ x \ e)$ now uses meta-language abstraction:
 $(\text{Recfun } \tau_1 \ \tau_2 \ (f. \ x. \ e))$.

Working Statically with HOAS

To Code

We're going to write code for an AST and pretty-printer for MinHS with HOAS.

Seeing as this requires us to **look under abstractions** without evaluating the term, we have to extend the AST with special “tag” values.

Static Semantics

To check if a MinHS program is well-formed, we need to check:

- 1 **Scoping** – all variables used must be well defined
- 2 **Typing** – all operations must be used on compatible types.

Static Semantics

To check if a MinHS program is well-formed, we need to check:

- ① **Scoping** – all variables used must be well defined
- ② **Typing** – all operations must be used on compatible types.

Our judgement is an extension of the scoping rules to include types:

Under this **context** of assumptions

$\Gamma \vdash e : \tau$

The expression is assigned this type

The **context** Γ includes **typing assumptions** for the variables:

$x : \text{Int}, y : \text{Int} \vdash (\text{Plus } x \ y) : \text{Int}$

Static Semantics

$$\frac{}{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \frac{}{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$

$$\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}$$

$$\frac{}{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$

$$\frac{}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : }$$

Static Semantics

$$\frac{}{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \frac{}{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$

$$\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}$$

$$\frac{}{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$

$$\frac{}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) :}$$

Static Semantics

$$\frac{}{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \frac{}{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$
$$\frac{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}}{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$
$$\frac{\Gamma \vdash e_1 : \text{Bool}}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \text{Int}}$$

Static Semantics

$$\frac{}{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \frac{}{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$
$$\frac{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}}{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$
$$\frac{\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \tau}$$

Static Semantics

$$\frac{}{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \frac{}{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$

$$\frac{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}}{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$

$$\frac{\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \tau}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash (\text{Var } x) : \tau}$$

Static Semantics

$$\frac{}{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \frac{}{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$

$$\frac{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}}{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$

$$\frac{\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \tau}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash (\text{Var } x) : \tau} \quad \frac{\Gamma, \quad \vdash e : \tau_2}{\Gamma \vdash (\text{Recfun } \tau_1 \ \tau_2 \ (f. x. e)) : \tau_1}$$

Static Semantics

$$\frac{}{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \frac{}{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$

$$\frac{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}}{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$

$$\frac{\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \tau}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash (\text{Var } x) : \tau} \quad \frac{\Gamma, x : \tau_1, \quad \vdash e : \tau_2}{\Gamma \vdash (\text{Recfun } \tau_1 \ \tau_2 \ (f. x. e)) : \tau_2}$$

Static Semantics

$$\overline{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \overline{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$

$$\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}$$

$$\overline{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$

$$\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau$$

$$\overline{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \tau}$$

$$(x : \tau) \in \Gamma$$

$$\overline{\Gamma \vdash (\text{Var } x) : \tau}$$

$$\Gamma, x : \tau_1, f : (\tau_1 \rightarrow \tau_2) \vdash e : \tau_2$$

$$\overline{\Gamma \vdash (\text{Recfun } \tau_1 \ \tau_2 \ (f. x. e)) : \tau_2}$$

Static Semantics

$$\frac{}{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \frac{}{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$

$$\frac{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}}{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$

$$\frac{\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \tau}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash (\text{Var } x) : \tau} \quad \frac{\Gamma, x : \tau_1, f : (\tau_1 \rightarrow \tau_2) \vdash e : \tau_2}{\Gamma \vdash (\text{Recfun } \tau_1 \ \tau_2 \ (f. x. e)) : \tau_1 \rightarrow \tau_2}$$

$$\frac{}{\Gamma \vdash (\text{Apply } e_1 \ e_2) : \tau}$$

Static Semantics

$$\frac{}{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \frac{}{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$

$$\frac{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}}{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$

$$\frac{\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \tau}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash (\text{Var } x) : \tau} \quad \frac{\Gamma, x : \tau_1, f : (\tau_1 \rightarrow \tau_2) \vdash e : \tau_2}{\Gamma \vdash (\text{Recfun } \tau_1 \ \tau_2 \ (f. x. e)) : \tau_1 \rightarrow \tau_2}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2}{\Gamma \vdash (\text{Apply } e_1 \ e_2) : \tau_2}$$

Static Semantics

$$\frac{}{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \frac{}{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$

$$\frac{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}}{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$

$$\frac{\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \tau}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash (\text{Var } x) : \tau} \quad \frac{\Gamma, x : \tau_1, f : (\tau_1 \rightarrow \tau_2) \vdash e : \tau_2}{\Gamma \vdash (\text{Recfun } \tau_1 \ \tau_2 \ (f. x. e)) : \tau_1 \rightarrow \tau_2}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash (\text{Apply } e_1 \ e_2) : \tau_2}$$

Static Semantics

$$\frac{}{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \frac{}{\Gamma \vdash (\text{Lit } b) : \text{Bool}}$$

$$\frac{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}}{\Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int}}$$

$$\frac{\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \tau}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash (\text{Var } x) : \tau} \quad \frac{\Gamma, x : \tau_1, f : (\tau_1 \rightarrow \tau_2) \vdash e : \tau_2}{\Gamma \vdash (\text{Recfun } \tau_1 \ \tau_2 \ (f. x. e)) : \tau_1 \rightarrow \tau_2}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash (\text{Apply } e_1 \ e_2) : \tau_2}$$

Static Semantics

$$\begin{array}{c}
 \overline{\Gamma \vdash (\text{Num } n) : \text{Int}} \quad \overline{\Gamma \vdash (\text{Lit } b) : \text{Bool}} \\
 \overline{\Gamma \vdash e_1 : \text{Int} \quad \Gamma \vdash e_2 : \text{Int}} \\
 \Gamma \vdash (\text{Plus } e_1 \ e_2) : \text{Int} \\
 \overline{\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau} \\
 \Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \tau \\
 \\
 \overline{(x : \tau) \in \Gamma} \quad \overline{\Gamma, x : \tau_1, f : (\tau_1 \rightarrow \tau_2) \vdash e : \tau_2} \\
 \Gamma \vdash (\text{Var } x) : \tau \quad \Gamma \vdash (\text{Recfun } \tau_1 \ \tau_2 \ (f. x. e)) : \tau_1 \rightarrow \tau_2 \\
 \overline{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1} \\
 \Gamma \vdash (\text{Apply } e_1 \ e_2) : \tau_2
 \end{array}$$

Let's implement a *type checker*.

Dynamic Semantics

Structural Operational Semantics (Small-Step)

Initial states:

Dynamic Semantics

Structural Operational Semantics (Small-Step)

Initial states: All well typed expressions.

Final states:

Dynamic Semantics

Structural Operational Semantics (Small-Step)

Initial states: All well typed expressions.

Final states: $(\text{Num } n)$, $(\text{Lit } b)$,

Dynamic Semantics

Structural Operational Semantics (Small-Step)

Initial states: All well typed expressions.

Final states: (Num n), (Lit b), **Recfun too!**

Evaluation of built-in operations:

$$\frac{e_1 \mapsto e'_1}{(\text{Plus } e_1 \ e_2) \mapsto (\text{Plus } e'_1 \ e_2)}$$

(and so on as per arithmetic expressions)

Specifying If

$$\frac{e_1 \mapsto e'_1}{(\text{If } e_1 \ e_2 \ e_3) \mapsto (\text{If } e'_1 \ e_2 \ e_3)}$$
$$\frac{}{(\text{If } (\text{Lit True}) \ e_2 \ e_3) \mapsto e_2}$$
$$\frac{}{(\text{If } (\text{Lit False}) \ e_2 \ e_3) \mapsto e_3}$$

How about Functions?

Recall that `Recfun` is a **final state** – we don't need to evaluate it unless it's applied to an argument.

Evaluating **function application** requires us to:

- ① Evaluate the left expression to get a `Recfun`;
- ② evaluate the right expression to get an argument value; and
- ③ evaluate the function's body, after supplying substitutions for the abstracted variables.

$$\frac{e_1 \mapsto e'_1}{(\text{Apply } e_1 \ e_2) \mapsto (\text{Apply } e'_1 \ e_2)}$$

$$\frac{e_2 \mapsto e'_2}{(\text{Apply } (\text{Recfun} \dots) \ e_2) \mapsto (\text{Apply } (\text{Recfun} \dots) \ e'_2)}$$

How about Functions?

Recall that `Recfun` is a **final state** – we don't need to evaluate it unless it's applied to an argument.

Evaluating **function application** requires us to:

- ① Evaluate the left expression to get a `Recfun`;
- ② evaluate the right expression to get an argument value; and
- ③ evaluate the function's body, after supplying substitutions for the abstracted variables.

$$\begin{array}{c}
 \dfrac{e_1 \mapsto e'_1}{(\text{Apply } e_1 \ e_2) \mapsto (\text{Apply } e'_1 \ e_2)} \\
 \dfrac{e_2 \mapsto e'_2}{(\text{Apply } (\text{Recfun } \dots) \ e_2) \mapsto (\text{Apply } (\text{Recfun } \dots) \ e'_2)} \\
 \dfrac{v \in F}{(\text{Apply } (\text{Recfun } \tau_1 \ \tau_2 \ (f.x. \ e)) \ v) \mapsto e[x := v, f := (\text{Recfun } \tau_1 \ \tau_2 \ (f.x. \ e))]}
 \end{array}$$